

Millions of people play games, but only a fraction dare to embark on the journey of "game development." Becoming a Game Developer is an alluring dream, where you combine creative artistry with technical logic to build entire virtual worlds.

However, with experience **programming training** We've noticed that the most common mistake beginners make is starting with an ambitious "open-world MMORPG" project. They quickly become overwhelmed by the workload and give up.

The truth is: Making games **very difficult** It requires a foundation. **programming** Solid foundation, logical thinking, perseverance, and a clear roadmap. This article, based on expertise and industry standards, will provide you with a detailed roadmap to conquering two industry giants: Unity and Unreal Engine.

Before you decide whether to install Unity or Unreal, you **Right** Laying the foundation is crucial. Skipping this step is like building a house without a foundation.

- **Core programming mindset** You can't make games if you don't know how. **programming** Learn the basic concepts (variables, data types, if/else statements, loops, functions) using any language (Python or C# are good choices).
- **Object-Oriented Programming:** This is absolute mandatory requirement The entire modern game development philosophy is based on OOP. Everything in the game (player, enemy, bullet) is an "Object" with properties and behavior. You must have a deep understanding of four properties: inheritance, encapsulation, polymorphism, and abstraction.
- **Basic mathematics and physics** You don't need to be a math professor, but you need understanding **Vector** to handle position, direction of movement, velocity, **Trigonometry** to calculate angles (e.g., camera rotation angle, bullet trajectory) and **Matrices** More advanced features for 3D graphics.
- **Git and GitHub Code Management Tools** Game projects are very susceptible to failure after just one test run. Git is a "time machine" that helps you save versions and revert to them when needed; it's also an essential tool for teamwork.



Unity (Popular and flexible):

- Programming languages: C# (C-Sharp)
- Difficulty(For beginners):More accessibleBecause C# is a modern, secure, and easier-to-learn language than C++.
- Strengths: 2D Games, Mobile Games (iOS/Android), VR/AR, Indie Games (independent), multi-platform.
- Community, forumExtremely large, with a vast amount of documentation, guides, and resources available.

Unreal Engine (AAA Graphics):

- Programming language: C++(Platform) andBlueprints(Visual script)
- Difficulty(for beginners)): Steeper.C++ is incredibly powerful but complex. However, Blueprints allows you to create games thatNo coding required..
- Strengths:AAA 3D games (top-notch graphics), console games (PS5, Xbox), architectural simulations, movies.
- Community, forum:Very large, but focused on high-end 3D.

Trainz's advice to you is:

If you're a complete beginner, want to make mobile games, 2D games, or want to get started.**programming** lightly **Please choose Unity**.

If you are passionate about high-end graphics, want to create AAA 3D games, or you**very embarrassed**Write code (you can start with Blueprints)**Please choose Unreal Engine**.

Feature	Unity	Unreal Engine	Godot	Construct
Learning Curve	Moderate	Steep	Gentle	Very gentle
Ideal For	2D/3D games, cross-platform	Graphically intensive games	2D/3D games, open-source	2D Games, no programming
Community Support	Extensive	Extensive	Growing	Strong
Best for Beginners?	Yes	With visual scripting	Yes	Yes

1. Getting acquainted with Unity Editor

Learn how to use the Unity interface and understand the "key" concepts such as:

- SceneYour game screen.
- GameObjectAll objects in the game (player, enemy, wall).
- Component: Functional "pieces" attached to a GameObject (e.g., Transform to contain position, Rigidbody to handle physics).
- ScriptThis is where you write C# code to control behavior.
- PrefabSample GameObjects for you to reuse.

Learn the core Unity APIs:

- Input: Retrieves information from the keyboard, mouse, and touch screen.
- Transform: Move, rotate, and resize objects.
- Physics (Rigidbody and Collider): To cause objects to fall or collide.
- Instantiate/DestroyCreates and destroys objects (bullets, explosion effects).

Make your first complete game (which is very important)

DON'T make a big game. Choose a "classic" game andcopyIt. For example: Flappy Bird, Pong, Pac-Man, or a simple platformer game. You will learn.entiregame development process

Learn advanced features

- UI (User Interface): Create menus, display scores.
- Animation: Making the character move.
- Audio: Add sound effects, background music.

2. Getting acquainted with Unreal Editor

You will learn about the UE interface and core concepts:

- LevelEquivalentSceneby Unity.
- ActorEquivalentGameObject(everything in the level).
- Component: Functional "pieces" (Static Mesh, Skeletal Mesh, Camera).
- Blueprint ClassThe combination ofActorand logicBlueprint.

Making games using Blueprints

Similar to Unity, start with a small project (e.g., a simple 3D side-scrolling game, a basic first-person shooter) to understand the process.

Switch to C++ (advanced step)

When you need maximum performance or complex logic handling, you'll need C++. You'll learn how C++ interacts with Blueprints (this is the greatest strength of the UE). Learn the core C++ UE APIs (UObject, AActor, Pawn, Character).

Learn advanced UE systems:

- UMG (Unreal Motion Graphics): UI design.
- Animation, Materials, and Niagara (explosion effects) systems.